

ARI 2129 · PRINCIPLES OF COMPUTER VISION

Histogram of Oriented Gradients



Capturing shape, one edge direction at a time.

Team 7 – Eytan Attard Vella, Julian Galea, Julian Grech, Gilmor Azzopardi, Kai James Azzopardi

Group Project · Learning Pack

Why do we need HOG?

THE PROBLEM

Raw pixels are unstable.

Lighting shifts, shadows, small movements, and colour changes can completely alter pixel values - even when the underlying object is identical.

Models that rely on exact pixel values *fail at the slightest variation.*

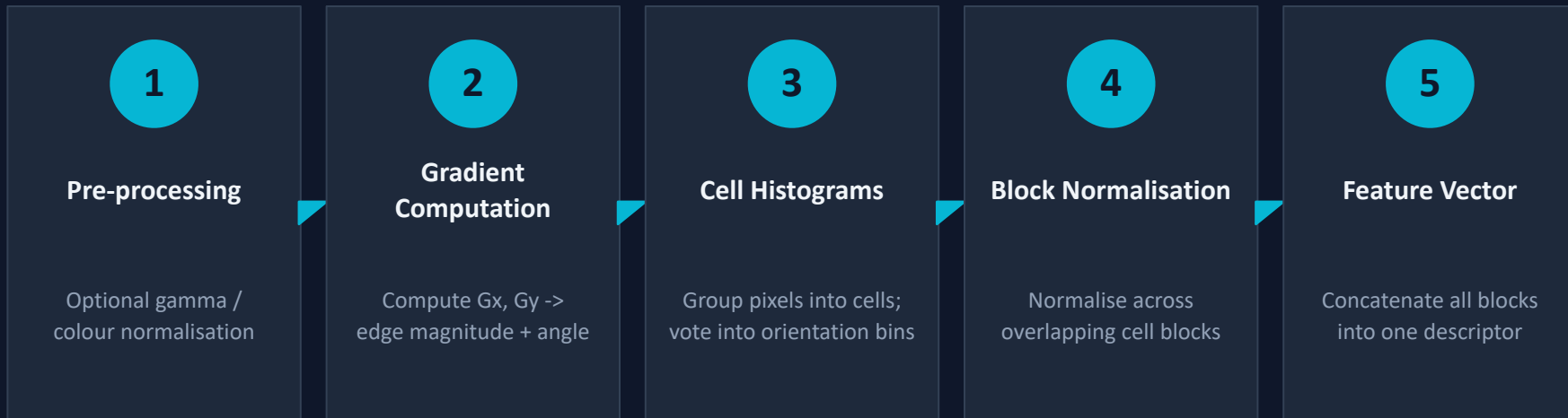
THE INSIGHT

Edges describe shape.

An object's outline persists even when lighting, colour, or texture changes.

HOG captures the **distribution of local edge directions** across the image - a compact structural fingerprint that ignores everything else.

The HOG Pipeline in 5 stages



Stages 2 -> 5 are the technical heart of HOG. We'll work through each one.

Gradient Computation

Edge filters

Convolve the image with the simple 1-D centred kernels:

Gx:

-1	0	1
----	---	---

 (horizontal change)

Gy:

-1
0
1

 (vertical change,
top -> bottom)

From (Gx, Gy) -> polar form

Magnitude (edge strength)

$$M = \sqrt{G_x^2 + G_y^2}$$

Orientation (edge angle)

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Unsigned gradients: angles map to 0°-180°. A dark -> light and light -> dark edge share the same orientation.

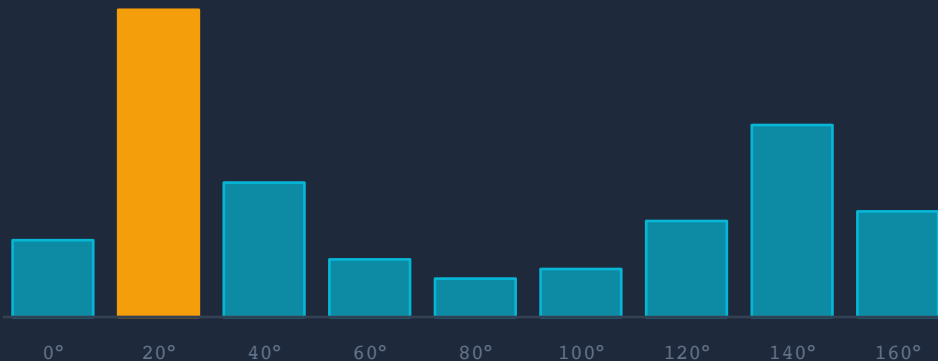
Cell Histograms - Spatial Binning

8×8 pixel cell



64 pixels -> one histogram

9-bin histogram (each bin = 20°)



Each pixel votes **into a bin** based on its orientation, weighted by its **magnitude**. Bilinear interpolation splits each vote between the two nearest bins.

Block Normalisation

Why normalise?

Same shape, different lighting -> different magnitudes.

Dark image: [2, 4, 6]

Bright image: [4, 8, 12]

Different magnitudes -> **identical ratios**. Normalisation keeps the pattern, drops the brightness.

L2-Hys normalisation (Dalal & Triggs)

$$v_{\text{norm}} = \frac{v}{\sqrt{||v||^2 + \epsilon^2}}$$

...then clip values at 0.2 and re-normalise (the 'Hys' step). This reduces the difference between the brightest and darkest values.

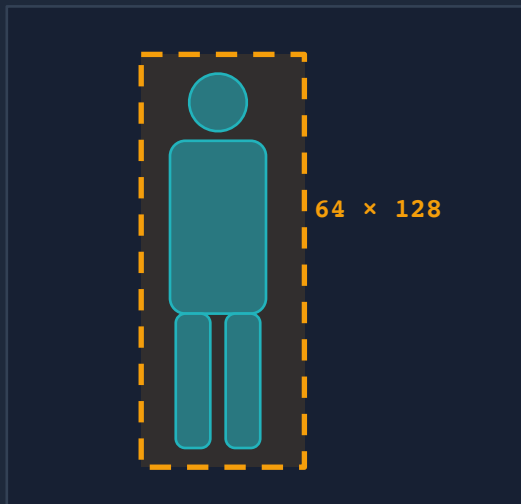
Overlapping 2x2 blocks (stride = 1 cell)



Same cell -> multiple blocks -> multiple normalisations within different areas -> more robust.

Feature Vector & Sliding Window

Sliding window (64 × 128 px)



The Dalal & Triggs descriptor

Cells per window $8 \times 16 = 128$

Blocks (2×2, stride 1) $7 \times 15 = 105$

Values per block $4 \text{ cells} \times 9 \text{ bins} = 36$

105 blocks × 36 values = **3,780** features per window

-> fed to a linear SVM classifier

From pixels to polar coordinates

3 × 3 patch · centre pixel highlighted

80	100	120
80	100	190
80	142	120

$G_x \rightarrow$ right - left

$G_y \rightarrow$ bottom - top

STEP 1 · Compute gradients

$$G_x = 190 - 80 = 110$$

$$G_y = 142 - 100 = 42$$

STEP 2 · Convert to polar

$$M = \sqrt{110^2 + 42^2} = \sqrt{13864} \approx 117.7$$

$$\theta = \arctan(42/110) \approx 20.9^\circ$$

One pixel $\rightarrow$ one vector: **magnitude 117.7** at **20.9°**

Voting and normalisation

STEP 3 · Bilinear vote split

Pixel at **20.9°** carries magnitude **117.7**.

It falls between bins 20° and 40° - split the vote by proximity.

$$\text{Bin } 20^\circ \text{ (near): } 117.7 \times (19.1/20) = \mathbf{112.4}$$

$$\text{Bin } 40^\circ \text{ (far): } 117.7 \times (0.9/20) = \mathbf{5.3}$$

Closer angle -> larger share of the magnitude.

STEP 4 · Sum into the cell histogram

$$H_{\text{cell}} = [15, \mathbf{250}, 40, 10, 5, 5, 10, 20, 15]$$

Peak at 20° reflects the strong horizontal edge.

STEP 5 · L2 normalise the block

Block $v = [H_1, H_2, H_3, H_4]$ -> 36 values

$$||v|| = \sqrt{\sum v^2} \approx \sqrt{260,400} \approx \mathbf{510}$$

Divide every value by ≈ 510

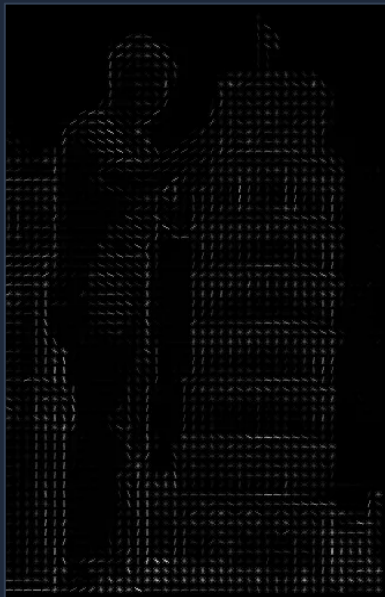
-> all entries fall between 0 and 1 -> lighting-invariant.

Parameter Choices & Trade-offs

PARAMETER	LOWER VALUE	HIGHER VALUE	CLASSIC CHOICE
Cell size	More detail · noisy	Smoother · less detail	8 × 8 px
Orientation bins	Poor angle discrimination	Precise but sparse / noisy	9 (unsigned)
Block size	Local contrast detail	Broader normalisation	2 × 2 cells
Block stride	More overlap · Bigger vector	Less Robust	stride = 1 cell
Window size	Local focus · less context	More context · noisier	64 × 128 px

HOG parameters trade off *detail* vs. *robustness* vs. *compute time*.
Dalal & Triggs's defaults are tuned for pedestrians at this aspect ratio.

Successful example



Parameters

- 9 bins
- 8x8 px per cell
- 2x2 (16x16px) cells per block

Image Credits: N. J. Karthika and C. Saravanan, "Addressing False Positives in Pedestrian Detection,"

Failure Example

Parameters

- 9 bins
- 8x8 px per cell
- 2x2 (16x16px) cells per block

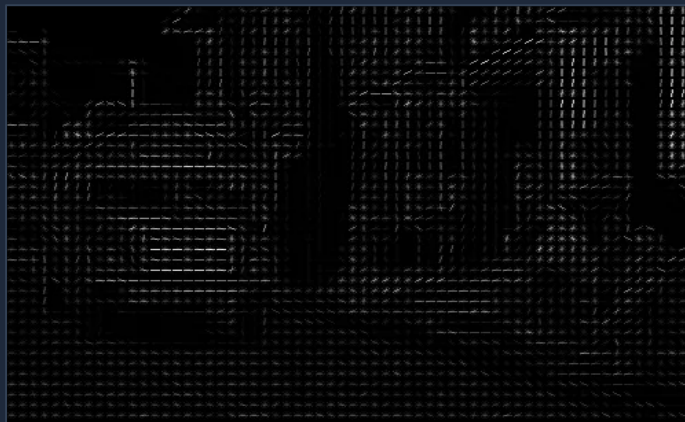


Image Credits: N. J. Karthika and C. Saravanan, "Addressing False Positives in Pedestrian Detection,"

Where HOG Breaks Down



Rotation

Fixed bins assume upright orientation. A lying pedestrian -> wrong bins, broken descriptor.



Scale

Fixed window size. Tiny or huge objects don't align with expected cell structure.



Occlusion / pose

Hidden parts replace true gradients with background ones. Articulation moves edges to wrong cells.



Background clutter

Waves, branches, railings can mimic object gradients -> false positives (HOGgles effect).



Lighting extremes

Darkness or overexposure corrupts gradients. Normalisation can't recover missing info.



No colour / texture

Pure shape descriptor. Mannequin, statue, and person can look identical to HOG.

When NOT to use HOG

Avoid HOG if you need: rotation/scale invariance · colour or texture cues · state-of-the-art accuracy required

CNNs

YOLO · Faster R-CNN

USE WHEN

Large dataset, GPU available, state-of-the-art accuracy required.

WHY

Learns colour, texture, and hierarchical features end-to-end.

SIFT / ORB

Local descriptors

USE WHEN

Matching the same object across rotated, scaled, or repositioned views.

WHY

Designed for invariance; sparse key-points scale better than dense HOG.

Real-World Applications

CASE STUDY

Pedestrian detection

in early ADAS (driver-assistance) systems

64 × 128 window

3,780-dim feature vector -> linear SVM

*Fast, deterministic, interpretable -
exactly what early in-car ECUs needed*

Other places HOG still runs today

- **Dlib face detector**

HOG + linear SVM at its core. Ships in OpenCV bindings, used in OpenFace and many accessibility tools.

TAKEAWAYS

HOG, in three sentences.

1

Shape, not pixels.

HOG describes images by where edges are and which way they point, discarding colour, lighting, and fine texture.

2

A mechanical pipeline.

Gradients -> cells -> blocks -> normalisation -> one long feature vector. Every step is interpretable and computable by hand.

3

Right tool, sometimes.

Replaced by CNNs at the frontier, but still the right choice for speed, interpretability, low compute, or small data.

Questions?